

TASK MANAGER

Web Application

A complete guide for students — what it is, how it works, and what every file does

01 — WHAT IS THIS APPLICATION?

This is a web-based Task Manager. Students and teachers can register an account, log in, and maintain a personal list of tasks. Each task has a title and a short description.

There are two kinds of users:

- Regular user — can only see and manage their own tasks.
- Administrator — can see and manage the tasks of every user in the system.

The application covers the four core operations of any database-driven system, known as CRUD:

Operation	Stands for	What it does in this app
Create	C	Add a new task or register a new user
Read	R	Display the list of tasks on screen
Update	U	Edit an existing task
Delete	D	Remove a task from the database

02 — TECHNOLOGIES USED

Technology	Role in this application
HTML	Defines the structure of every page and form the user sees
CSS	Controls colors, typography, layout, and visual style
PHP	Runs on the server — receives form data, queries the database, and generates dynamic pages
SQL	The language used to query and update the database (SELECT, INSERT, UPDATE, DELETE)
MySQL	The database engine that stores all users and tasks permanently

03 — DATABASE STRUCTURE

The application requires a database named tasks containing two tables.

Table: users

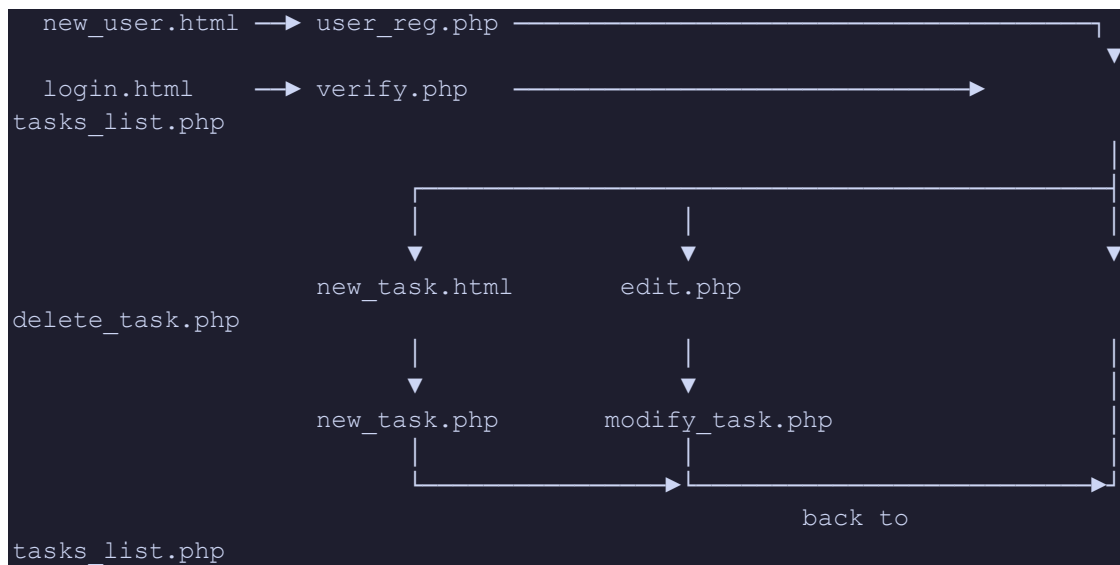
Column	Type	Description
iduser	INT (PK, auto)	Unique numeric ID for each user
firstname	VARCHAR	User's first name
lastname	VARCHAR	User's last name
username	VARCHAR	Login name (must be unique)
password	VARCHAR	User's password
admin	INT (0 or 1)	0 = regular user 1 = administrator

Table: task

Column	Type	Description
id	INT (PK, auto)	Unique numeric ID for each task
title	VARCHAR	Short title of the task
description	TEXT	Longer description of the task
iduser	INT (FK → users)	ID of the user who owns the task
created_at	DATETIME	Date and time the task was created (set automatically)

04 — HOW PAGES CONNECT TO EACH OTHER

The diagram below shows the path a user follows from the moment they open the app until they manage their tasks.



05 — EVERY FILE EXPLAINED

db.php [PHP]

The backbone of the application. It opens the connection to the MySQL database and starts a PHP session so the app can remember which user is logged in as they navigate between pages.

- Included by every PHP file that needs database access via `include("db.php")`
- Contains: server host, database username, password, and database name
- This is the only file you need to edit when deploying to a new server

login.html [HTML]

Displays the login form with two fields: username and password. When the user clicks Login, the data is sent to `verify.php` using the HTTP POST method.

verify.php [PHP]

Receives the credentials from the login form and checks them against the users table. Three possible outcomes:

- SQL error → redirects to `query_error.html`
- User not found → redirects to `error_login.html`
- Valid credentials → saves user ID and admin flag in session variables, redirects to `tasks_list.php`

new_user.html [HTML]

Registration form for new accounts. Fields: first name, last name, username, password, and an "Administrator" checkbox. Submits to `user_reg.php`.

user_reg.php [PHP]

Handles new user registration. First checks whether the chosen username already exists in the database:

- Username taken → redirects to `user_exist.html`
- Username free → inserts the new user into the users table and redirects to `tasks_list.php` (user is immediately logged in)

tasks_list.php [PHP + HTML]

The main screen after login. Displays tasks in a table. The SQL query depends on the user role:

- Regular user: shows only their own tasks (SQL JOIN filtered by session user ID)
- Administrator: shows all tasks from all users
- Each row has an Edit button (pencil icon) and a Delete button (bin icon)
- A "New Task" button navigates to `new_task.html`

new_task.html [HTML]

Simple form with two fields — Title and Description. Submits to new_task.php via POST.

new_task.php [PHP]

Receives the title and description, reads the current user's ID from the session, and executes an INSERT query to save the new task. Redirects back to tasks_list.php.

edit.php [PHP + HTML]

Receives the task ID via the URL (GET parameter), fetches the current title and description from the database, and pre-fills a form so the user can modify them. Submits to modify_task.php.

modify_task.php [PHP]

Receives the updated title and description and executes an UPDATE query to save the changes. Redirects back to tasks_list.php.

delete_task.php [PHP]

Receives the task ID via the URL (GET parameter) and executes a DELETE query to remove it from the database. Redirects back to tasks_list.php.

style.css [CSS]

Defines the visual appearance of the entire application: dark panel backgrounds, blue accent colors, form and table layout, button styles and hover effects, and the full-page background image.

Informational pages (no logic — display only)

File	When it appears
error_login.html	Username or password entered on the login form does not exist in the database
user_exist.html	Someone tries to register a username that is already taken
query_error.html	A PHP file encounters a SQL syntax error
welcome.html	Welcome screen (available but not actively used in the main flow)

images/ folder

File	Used by	Purpose
wall.png	style.css	Full-page background texture
edit.png	tasks_list.php	Icon for the Edit button on each task row
delete.png	tasks_list.php	Icon for the Delete button on each task row